

AURA

Pearls AQI Predictor

Project Documentation

10Pearls Shine Internship Program — Data Science Track

Ali Hassan

August 2026

Table of Contents

1. Introduction	3
2. System Architecture	3
2.1 Pipeline Stages	3
2.2 Scheduling	3
3. Data & Feature Engineering	4
3.1 Data Source Selection	4
3.2 Feature Store Design	4
3.3 Feature Selection	4
3.4 Preprocessing	5
4. Modeling	5
4.1 Methodology	5
4.2 Initial Model Comparison	5
4.3 Per-Horizon Modeling	5
5. Explainability (SHAP)	6
6. Web Application	6
6.1 Backend	6
6.2 Frontend	7
6.3 Design: The Aurora Theme	7
6.4 Hazardous AQI Alerting	7
7. MLOps & Automation	7
7.1 Compute Quota Management	7
8. Challenges & Engineering Decisions	8
8.1 Data & Feature Pipeline	8
8.2 Hopsworks Integration	8
8.3 Automation	8
9. Results Summary	8
10. Limitations & Future Work	9
11. Conclusion	9

1. Introduction

Air quality in Lahore is a recurring public health concern, and reliable short-term forecasts give residents and institutions a practical way to plan around poor-air days rather than react to them after the fact. Aura is a fully serverless, end-to-end machine learning system that forecasts Lahore's Air Quality Index (AQI) up to four days ahead, built as the capstone project for the 10Pearls Shine Internship Program (Data Science track).

The internship brief required a complete pipeline: automated data ingestion, feature engineering, a comparison between statistical and deep learning modeling approaches, model explainability, a hazardous-AQI alerting mechanism, and a working dashboard — all built without a persistently hosted server. This document explains what was built, the reasoning behind each major decision, the problems encountered along the way, and what remains open.

Required deliverables for the program were a GitHub repository and this documentation.

2. System Architecture

The system follows the FTI (Feature / Training / Inference) pipeline pattern: feature computation, model training, and inference are kept as separate, decoupled stages that communicate only through the Hopsworks Feature Store and Model Registry, rather than passing data directly between stages. This separation is what makes the "100% serverless" requirement practical — each stage can run independently, on its own schedule, without any of them needing to stay resident.

2.1 Pipeline Stages

Stage	Responsibility	Reads From	Writes To
Feature Pipeline (hourly)	Fetches and parses Open-Meteo air quality + weather data for Lahore	Open-Meteo API	<code>raw_hourly_readings</code> (Feature Store)
Engineered Features Pipeline (daily)	Aggregates the prior 7 days of raw data into lag/rolling/calendar features	<code>raw_hourly_readings</code>	<code>daily_engineered_features</code> (Feature Store)
Recovery Pipelines (daily/weekly/monthly)	Re-ingests recent complete periods to repair any gaps	Open-Meteo API	<code>raw_hourly_readings</code> / <code>daily_engineered_features</code>
Model Training Pipeline (daily)	Trains the per-horizon MLP models and the RobustScaler	<code>daily_engineered_features</code>	Hopsworks Model Registry
Backend (FastAPI)	Serves predictions on request	Model Registry, <code>daily_engineered_features</code>	JSON API responses
Frontend (React + Vite)	Displays the forecast dashboard	Backend API	Rendered dashboard

2.2 Scheduling

All scheduled stages run via GitHub Actions, chosen specifically because it is serverless by nature. The alternative considered was Apache Airflow, which was ruled out because it requires a persistently hosted server to run its scheduler — a direct conflict with the project's serverless requirement. Six original workflows (hourly ingestion, daily engineered features, daily recovery, weekly recovery, weekly engineered recovery, monthly engineered recovery) were later joined by a seventh (daily model training).

3. Data & Feature Engineering

3.1 Data Source Selection

The initial data source plan used AQICN for air quality and OpenWeather for weather data. Both were dropped for the same underlying reason: neither offered free access to historical pollutant/weather data, which is required to build any meaningful training set. AQICN's Lahore station data was also found to be stale and missing several pollutant readings during initial testing.

Open-Meteo was adopted as the sole data source, providing both an Air Quality API and a Forecast API, with genuine free historical access. One constraint was discovered during backfilling: Open-Meteo's underlying CAMS Global Atmospheric Composition dataset only provides data for Lahore from August 2022 onward, since Lahore falls outside the Europe-only CAMS reanalysis coverage that extends further back. The historical backfill window was set accordingly.

3.2 Feature Store Design

Two Hopsworks Feature Groups separate the permanent raw record from the model-consumable features:

- **raw_hourly_readings** — one row per hour, untouched pollutant and weather values, the permanent historical record.
- **daily_engineered_features** — one row per day, built by aggregating the prior 7 days of raw hourly data into lag features, 3- and 7-day rolling statistics, and calendar features. This is what the training and inference pipelines actually consume.

Features were tiered by expected importance ahead of formal validation: primary features (AQI, PM2.5, PM10, O₃) received full treatment — same-day statistics, 3-day lags, and 3-/7-day rolling mean and standard deviation. Secondary features (temperature, humidity, wind speed, pressure, dew point) received same-day value plus a 1-day lag. Minor features (CO, SO₂, NO₂) received only their same-day value. This produced 49 input columns feeding into feature selection.

3.3 Feature Selection

Sequential feature-group ablation was run — baseline, then AQI temporal features, primary pollutant temporal features, weather features, and calendar features — each evaluated against the previous best on validation RMSE, MAE, and R², using RFECV (Recursive Feature Elimination with Cross-Validation) to guide selection within each group. RFECV recommendations were followed when they improved validation performance and preserved consistent treatment within a feature group, and overridden

when they did not — for example, RFECV suggested dropping only part of PM10's temporal block while keeping the rest, which was rejected in favor of consistent treatment across PM2.5, PM10, and O₃.

Weather features were tested and rejected outright — they did not improve validation metrics even after RFECV-guided reduction. Calendar features were tested and accepted, with RFECV reducing the group down to a single feature, month, which did improve validation performance. The resulting final set is locked in 23 features, down from the original 49.

3.4 Preprocessing

A feature distribution analysis on the 23 selected features found wide variation in scale (for example, `month` has a range of 11, while `co_today`'s range exceeds 7,500), and most features were right-skewed, with several strongly so. This skew was treated as genuine data character rather than an artifact to correct. RobustScaler was selected for Ridge Regression and the Dense Neural Network, chosen specifically for its reduced sensitivity to skew and outliers; Random Forest used raw, unscaled values. A log1p transform was applied ahead of scaling to the most heavily skewed features: `co_today` and the PM2.5/PM10 rolling standard deviation features.

4. Modeling

4.1 Methodology

Data was split chronologically into train, validation, and test sets (70/15/15) with no shuffling, to avoid leaking future information into training — a standard requirement for time-series data.

4.2 Initial Model Comparison

Random Forest, Ridge Regression, and a Dense Neural Network (MLP) were each tuned on the shared 23-feature set, evaluated by mean R² across all four forecast horizons:

Model	Configuration	Mean R ²
Random Forest	800 trees, depth 20, split/leaf 7, feature/sample subsampling	0.5616
Ridge Regression	alpha = 0.01	0.5818
Dense NN (MLP)	32 units, L2 + dropout	0.5866

Across all three model types, Day 1 forecasts were consistently strong (approximately 0.84 R²), while Days 2–4 sat in a much weaker 0.42–0.54 range and dragged down the mean. This gap motivated a shift away from one shared feature set and shared model configuration across all four horizons, toward independent feature selection and tuning per forecast day.

4.3 Per-Horizon Modeling

Each forecast horizon (Day 1 through Day 4) was treated as its own modeling problem, starting from the full 49-feature set rather than the shared 23. For each horizon, RFECV using Ridge Regression as the estimator was run as a starting point to surface which features carried signals for that specific horizon —

not as a rule to apply automatically. Selections were then manually iterated on and refined beyond RFECV's raw output for each horizon individually.

For Day 1, Random Forest, Ridge Regression, and the MLP were all evaluated; the MLP, tuned to 128 units with L2 regularization ($1e-3$), a learning rate of $5e-3$, and Huber loss, produced the best result of any model or configuration tested in the project — the first to clear the 0.90 R^2 target. This architecture and approach was carried forward to Days 2–4 rather than re-testing Random Forest and Ridge at every horizon.

Horizon	RMSE	MAE	R^2
Day 1	12.52	9.33	0.901
Day 2	25.59	18.23	0.593
Day 3	28.89	21.11	0.509
Day 4	29.59	21.77	0.506

Day 1 accuracy is strong and reliable. Accuracy declines for Days 2–4, which reflects the genuine difficulty of longer-horizon air quality forecasting rather than a modeling shortfall — this is reported plainly rather than obscured.

5. Explainability (SHAP)

SHAP (SHapley Additive exPlanations) was used to explain each of the four per-horizon MLP models, via `shap.DeepExplainer` with a 100-sample background set drawn from training data. For each model, a beeswarm plot, a bar plot and a waterfall plot of mean absolute SHAP values were generated.

Day 1: `pm25_today` is the dominant driver by a clear margin; `pm25_roll_mean_3`, `pm10_today`, and select AQI/PM2.5 lag features are secondary; weather and minor gas features have limited impact.

Day 2: `pm25_today` and `pm25_roll_mean_7` dominate (mean $|SHAP| \approx 19.5$ and ≈ 13.8); `aqi_today`, `co_today`, `aqi_lag_1`, and `o3_today` form the secondary tier.

Day 3: `pm25_roll_mean_7` (≈ 18.8) and `pm25_today` (≈ 14.3) lead, followed by `aqi_today` (≈ 11.2); several AQI/PM10 lag features show inverse relationships.

Day 4: `pm25_roll_mean_7` (≈ 14.8) and `aqi_today` (≈ 9.0) dominate; `pm25_lag_2` and `pm10_roll_mean_7` form the next tier.

Across all four horizons, PM2.5 — either its same-day value or its rolling mean — is the consistent top driver of the forecast. Its exact form shifts from the same-day value at Day 1 toward the 7-day rolling mean as the horizon lengthens, consistent with longer-horizon forecasts leaning more on smoothed trend than on the latest single reading.

6. Web Application

6.1 Backend

The backend is a FastAPI application, structured as an app package with a services layer (`feature_service`, `model_service`, `prediction_service`), a routes layer (the prediction route), and a schemas layer (the `prediction` schema). FastAPI was chosen over Flask, which the brief also permitted, for its async support, Pydantic-based request/response validation, and automatically generated API documentation. The backend and frontend communicate via environment-variable-configured URLs rather than hardcoded local defaults, and the backend is started via a single project-root entry point.

Two additional endpoints, `current_air_quality` and `current_weather`, were later added following the same services/routes/schemas structure, serving the dashboard's Statistics page.

Two further route groups were added later: raw and engineered historical backfill (`raw_historical_backfill`, `engineered_historical_backfill`), which bring the two Hopsworks feature stores up to date on demand rather than relying solely on scheduled automation, and `model_training`, which triggers a retrain against the latest available data. The two backfill routes share a common request/response schema and a dedicated utils module for shared backfill logic and route-level error handling. Each determines its working date range by reading the latest stored timestamp from the relevant feature store and backfilling from there to the current date (raw) or the current date minus one day (engineered) — the two routes intentionally use different end-date logic, reflecting the different granularity of each store.

6.2 Frontend

The frontend is built with Vite, TypeScript, and React (React Compiler enabled), using `npm` and `oxlint`. React was chosen over Streamlit or Gradio — both permitted by the brief — for full design control over a custom visual identity, since a templated dashboard framework would not support the design goals below.

The frontend was later restructured around `react-router-dom` to support multiple pages: a Prediction page (the original forecast dashboard) and a Statistics page, showing live current AQI, today's prediction, per-pollutant readings, and current weather conditions, each timestamped.

A third page, Backfill, was added to let the user trigger an on-demand data recovery and model retrain directly from the dashboard when the feature stores or predictions are behind. Its interface centers on a single action button — floating continuously, glowing on hover — paired with a progress bar using the same aurora color treatment as the background, showing live percentage progress and state text that fades between stages (raw backfill → engineered backfill → model training → complete). The Prediction and Statistics pages were each updated to detect stale prediction data and surface a notification card with a direct link to the Backfill page instead of silently showing outdated numbers.

6.3 Design: The Aurora Theme

The dashboard's visual identity is built around an aurora motif — a direct reference to the project name (Aura, short for Aurora) and to the visual concept of a flickering, living field of light. The background is implemented as stacked horizontal waves that drift individually (up/down and left/right) while the wave composition as a whole also moves up and down, approximating the aurora's constant, ambient motion. The sidebar uses a dark, glass-and-ice styling with a cracked-ice texture and light refraction effect, and, page header and prediction cards use a heavier-than-typical glassmorphism treatment (higher opacity, strong background blur) specifically to keep card text readable against the animated background.

6.4 Hazardous AQI Alerting

Rather than a separate static alert banner, hazardous AQI alerting is built directly into the prediction cards: each card's hue, symbol, and border/symbol color change dynamically based on that day's AQI severity level. This satisfies the brief's alerting requirement while keeping the alert tied directly to the specific forecast it concerns, rather than as a generic page-level notice.

7. MLOps & Automation

Hopsworks was selected as the feature store and model registry, chosen for its genuine free tier and its native fit for the FTI pipeline pattern, avoiding the need for a billing account. The Model Registry stores each trained MLP model together with its corresponding RobustScaler as a paired artifact, ensuring inference always applies the exact scaling the model was trained against.

Duplicated environment-setup logic across the growing set of workflow files was consolidated into two GitHub Actions composite actions: `setup-python-env` (checkout, Python setup, dependency installation) and `run-pipeline` (running a named script with the required secrets and environment configured).

7.1 Compute Quota Management

Partway through development, the combination of hourly ingestion and multiple recovery pipelines was found to be consuming Hopsworks' free-tier compute quota faster than anticipated — approximately 80% of the monthly allowance used before the month was over. Rather than risk exhausting the quota mid-project, all automated workflows were intentionally paused. During the pause, every pipeline was run and verified manually, and both feature stores were kept current through manual backfill rather than scheduled automation. Automation is re-enabled for submission, so the full scheduled pipeline runs live for evaluation. This was a deliberate resource-management decision made under a real constraint, not a gap left unaddressed.

7.2 Deployment

The frontend was deployed to Vercel, with the project's root directory set to `frontend/` to support the monorepo layout. The backend was deployed to FastAPI Cloud, the official hosting platform from the FastAPI maintainers, selected after two alternatives were ruled out: Hugging Face Spaces' Docker SDK, found to now require a paid billing account with no free tier available, and Render, whose signup required card verification with no accepted payment method available.

Since FastAPI Cloud resolves dependencies via `uv` rather than `pip`, a separate `pyproject.toml` and `uv.lock` were introduced specifically for this deployment target, while `requirements.txt` remains the source of truth for local development and all GitHub Actions pipelines.

A `vercel.json` was added at the frontend root after React Router routes returned 404 on direct navigation/refresh under Vercel's default routing.

8. Challenges & Engineering Decisions

A number of real issues were found and fixed over the course of the project. A representative selection, by area:

8.1 Data & Feature Pipeline

- Historical data ceiling: Open-Meteo's CAMS dataset only covers Lahore from August 2022 onward (the city falls outside the longer-running Europe-only CAMS reanalysis). The backfill window was set to start from the earliest available date rather than an arbitrarily longer one.
- Exception handling in the Open-Meteo response parser originally aborted an entire batch on a single bad hour; changed to skip only the affected hour so a single bad reading does not discard an otherwise-valid batch.

8.2 Hopsworks Integration

- On Windows, `hopsworks.login()` failed due to Linux-specific certificate paths used internally by the SDK; resolved with a custom Windows-compatible certificate folder. A related issue from hardcoded `/tmp` paths during feature insertion was resolved by patching the installed SDK to use a valid temporary directory.
- Feature Group insert jobs reported "success" in the Hopsworks UI while silently failing on first insert. The root cause was automatic statistics computation enabled by default on schema creation, which threw an internal server error; disabling statistics computation on the Feature Group resolved it.
- Hopsworks pins an older protobuf version than TensorFlow requires. This conflict never surfaced locally, since the working local environment was never fully re-resolved from a clean state — but GitHub Actions' clean installs hit it every time, with pip's resolver refusing to install both together. Resolved by installing Hopsworks separately with `--no-deps` after the rest of `requirements.txt`, so protobuf resolves against TensorFlow's requirement first, and Hopsworks is layered on top without pip re-checking its own declared constraint.
- The same protobuf conflict recurred under `uv`-based dependency resolution for the FastAPI Cloud deployment target, where `--no-deps` has no direct equivalent. Resolved using `uv`'s `dependency-metadata` override to declare Hopsworks' own dependency list as empty, forcing protobuf to resolve against TensorFlow's requirement instead, matching the same effect as the pip approach.

9. Results Summary

The completed system meets the internship brief's core requirements: a fully serverless, automated pipeline — with an on-demand backfill and retrain flow available directly from the dashboard as a supplement to scheduled automation — a statistical-versus-deep-learning model comparison (Random Forest and Ridge Regression against a Dense Neural Network); SHAP-based explainability per forecast horizon; a hazardous-AQI alerting mechanism built into the dashboard; and a working web application. The final per-horizon MLP configuration reaches an R^2 of 0.901 on Day 1, with Days 2–4 in the 0.51–0.59 range — reported honestly as the genuine difficulty ceiling of multi-day air quality forecasting on the available data volume, rather than adjusted to look stronger than it is. Both the frontend and backend are deployed and live for evaluation.

10. Limitations & Future Work

- Single city only: the model and feature store currently support Lahore exclusively. A city-selection page is scoped in the dashboard's navigation but not yet built, pending a feasibility check on whether sufficient historical data can be backfilled on demand for an arbitrary new city.
- Forecast accuracy declines with horizon length (Day 1 R^2 of 0.901 versus 0.51–0.59 for Days 2–4), a genuine characteristic of the forecasting problem rather than an oversight.
- An LSTM model was scoped as a stretch goal in the original plan but not pursued, given the limited data volume relative to the tuning cost; the Dense Neural Network alone satisfied the brief's statistical-versus-deep-learning comparison requirement.

11. Conclusion

Aura is a complete, serverless, end-to-end air quality forecasting system for Lahore — spanning automated data ingestion, a two-tier feature store, per-horizon deep learning models, SHAP-based explainability, and a custom-built dashboard — developed under real constraints, including a data source migration forced by free-tier limitations, a compute quota that required a deliberate pause-and-resume decision, and a modeling redesign midway through development once the initial shared-feature-set approach showed clear per-horizon weaknesses. Each of these is documented here as it actually happened, including the parts that didn't work on the first attempt.